



Final Year Project Report

ExportEdge: AI Screening for Global Fruit Markets

by

Muhammad Moosa – 405243

Hannan Yousaf Butt – 405326

Muhammad Ammar bin Akram – 414563

Supervisor: Dr. Shahzad Younis

Co-Advisor: Dr. Momina Moetesum

Department of Computing (DoC)

School of Electrical Engineering and Computer Sciences (SEECS)

National University of Sciences and Technology (NUST)

Islamabad, Pakistan

DECLARATION

We hereby declare that this project report entitled "*ExportEdge AI: AI Screening for Global Fruit Markets*" submitted to the Department of Computing (DoC), School of Electrical Engineering and Computer Science (SEECS), National University of Sciences and Technology (NUST) is a record of original work done by us under the guidance of our advisor Dr. Shahzad Younis and co-advisor Dr. Momina Moetesum. No part of this work has been plagiarised without proper citations. This project is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

Team Members

Hannan Yousaf Butt (405326)

Signature

Muhammad Moosa (405243)

Muhammad Ammar bin Akram (414563)

Advisor:

Dr. Shahzad Younis

Signature:

Co Advisor:

Dr. Momina Moetesum

Signature:

Date:

Place:

DEDICATION

To our parents, whose patience kept us going on the long nights when training jobs failed at 3 a.m., to Dr. Shahzad Younis and Dr. Momina Moetesum who pushed us to ask better questions, and to every researcher who has ever struggled to find quality agricultural data without access to expensive, large-scale inspection equipment.

ACKNOWLEDGEMENTS

We are deeply grateful to our advisor, Dr. Shahzad Younis, for the weekly guidance, the candid critique of our prototypes, and the steady encouragement to keep iterating when results were not where we wanted them. We thank our co-advisor, Dr. Momina Moetesum, for the technical direction on AI-driven quality inspection and export intelligence, and for the consistent support throughout the development of ExportEdge AI. We are also thankful to the KG Committee members, Dr. Fahad Ahmed Satti and Dr. Gibrail Islam, for their detailed reviews of our scope and design.

We acknowledge the support of the SEECS faculty and staff, particularly the FYP coordination team, for keeping the timeline structured and the deliverables clear. Finally, we thank our families and our classmates, whose feedback during demos shaped the user interface into something that is actually usable.

Table of Contents

1. Background	8
1.1. Problem Statement	8
1.2. Literature Review	8
1.3. Business Opportunity	9
1.4. Proposed Solution	9
1.5. Project Scope	10
1.6. Stakeholders	10
1.7. Technology Stack	10
2. Software Requirements Specifications	11
2.1. Requirements Elicitation Process	11
2.2. Use Case Diagram	12
2.2.1. User Scenarios	12
2.3. Functional Requirements	14
2.4. Non-Functional Requirements	22
2.4.1. Performance Requirements	22
2.4.2. Safety Requirements	22
2.4.3. Security Requirements	23
3. Software Design Specifications	24
3.1. Architectural Style	24
3.1.1. Data Acquisition and Persistence Layer	24
3.1.2. AI Analysis Layer	24
3.1.3. Intelligence Layer	24
3.1.4. Frontend Layer	25
3.2. Selection of Design Methodology	26
3.2.1. Simplicity	26
3.2.2. Modularity	26
3.2.3. Scalability	26
3.2.4. Flexibility	26
3.3. Data Representation Diagram	26

3.4.	Process Flow.....	27
3.5.	Design Models.....	28
3.5.1.	Data Flow Diagrams	28
3.5.2.	Sequence Diagrams	32
3.5.3.	State Transition Diagrams.....	35
3.5.4.	Class Diagram	38
4.	System Testing	40
4.1.	Unit Testing	40
4.2.	Integration Testing	41
4.3.	System Testing	42
4.4.	Performance Testing	43
5.	Deployment and System Integration	44
5.1.	Deployment Details	44
5.2.	System Integration Details.....	44
6.	User Interfaces	46
6.1.	Landing Page	46
6.2.	Home Page	47
6.3.	Processing Page	47
6.4.	Chatbot Page.....	48
7.	Conclusion	49
8.	References	51

List of Figures

Figure 2.1: Use Case Diagram for ExportEdge	12
Figure 3.1: Architecture Diagram for ExportEdge	25
Figure 3.2: ERD for ExportEdge	27
Figure 3.3: Process Flow Model for ExportEdge	28
Figure 3.4: Context Diagram for ExportEdge.....	29
Figure 3.5: DFD Level0 for ExportEdge	30
Figure 3.6: DFD Level1 for Analyze Mango Quality.....	31
Figure 3.7: DFD Level1 for Orchestrate and Generate Report.....	32
Figure 3.8: Sequence Diagram for Automated Mango Processing Pipeline.....	33
Figure 3.9: Sequence Diagram for Frontend Report Retrieval	34
Figure 3.10: Sequence Diagram for User Query and LLM Response	34
Figure 3.11: State Machine for Packhouse sorting controller.....	36
Figure 3.12: State Machine for Intelligence Orchestrator	38
Figure 3.13: Class Diagram for ExportEdge.....	39
Figure 6.1: Landing page for ExportEdge	46
Figure 6.2: Landing Page (continued) for ExportEdge.....	46
Figure 6.3: Home Page for ExportEdge.....	47
Figure 6.4: Processing Page for ExportEdge	47
Figure 6.5: Processing Page (results) for ExportEdge	48
Figure 6.6: Chatbot Page for ExportEdge.....	48

1. Background

1.1. Problem Statement

Pakistan is one of the world's leading mango producers, exporting hundreds of thousands of tons annually to markets across Europe, North America, and the Middle East. Despite this significant export volume, the industry suffers from substantial post-harvest losses estimated at 25–40%, largely attributable to inadequate quality control mechanisms at the packhouse level. Current inspection processes rely on manual visual grading, which is inherently inconsistent, labor-intensive, and incapable of detecting early-stage disease or subtle surface defects that lead to rejection at destination ports.

The absence of automated, AI-driven quality inspection at the point of export creates a critical gap: mangoes that appear acceptable to the human eye may carry fungal or bacterial infections that become apparent only after shipping, resulting in cargo rejection, financial loss, and reputational damage to Pakistani exporters. Furthermore, exporters lack access to real-time market intelligence that could help them direct their produce to the most profitable and regulation-compliant destination markets. These combined challenges, inconsistent quality grading and absence of export advisory tools, represent a significant barrier to maximizing Pakistan's agricultural export potential.

1.2. Literature Review

Several studies have explored the application of computer vision and deep learning for agricultural quality assessment. Saleem and Ahmad [2] demonstrated that Convolutional Neural Networks (CNNs) can effectively detect mango ripeness and quality grades using image data, achieving high classification accuracy under controlled conditions. Similarly, Zhang [3] applied CNN-based classification on the Fruit-360 dataset, demonstrating the feasibility of automated fruit grading systems. Munir et al. [4], in a NUST-published study, further validated the applicability of deep neural networks for fruit classification in Pakistani agricultural contexts.

Beyond classification, disease segmentation using architectures such as DeepLabV3 has been explored for precision agriculture applications. DeepLabV3 with a MobileNetV2 backbone has shown strong performance for semantic segmentation tasks on resource-constrained devices, making it suitable for edge deployment.

In the domain of export advisory and market intelligence, Retrieval-Augmented Generation (RAG) has emerged as an effective paradigm for grounding large language model responses in domain-specific knowledge bases. By combining vector similarity search with LLM generation, RAG systems can answer complex regulatory and pricing queries without requiring full model fine-tuning.

While prior work addresses individual components, classification, segmentation, or advisory, no existing system integrates all three into a unified, edge-deployed inspection pipeline tailored to the mango export sector. ExportEdge AI addresses this gap.

1.3. Business Opportunity

Pakistan's mango export industry generated approximately \$68 million USD in export revenue in recent years, with major markets including the United Kingdom, UAE, and Germany. The persistent problem of quality-related cargo rejections, estimated to affect 15–20% of export shipments, represents a direct and quantifiable financial loss. An automated inspection system that reduces rejection rates by even 10% could recover millions in lost revenue annually.

Beyond loss prevention, the integration of real-time market intelligence, including live price data from the UN Comtrade database and country-specific regulatory compliance guidance, enables exporters to make data-driven decisions about destination markets, potentially increasing realized export prices by directing higher-quality produce to premium markets.

ExportEdge AI targets the packhouse stage of the export supply chain, where mangoes are sorted and prepared for shipment. At this stage, automated AI inspection is both technically feasible and economically impactful, as decisions made here directly determine export eligibility and destination.

1.4. Proposed Solution

ExportEdge AI is an integrated agri-tech platform that combines computer vision, edge AI deployment, and large language model intelligence to automate mango quality inspection and export advisory. The system comprises three core components working in a sequential pipeline:

The first component is the AI Analysis Pipeline, deployed locally on a Raspberry Pi 5 edge device. A MobileNetV2-based classification model, converted to TensorFlow Lite format, classifies each mango frame as healthy or diseased with confidence scoring. For all mangoes, a DeepLabV3-based segmentation model, also converted to TFLite, generates pixel-level disease masks and computes disease area percentages. An OpenCV-based defect detection module further analyzes surface quality metrics including color uniformity and surface integrity scores.

The second component is the Export Intelligence Module, which integrates classification and segmentation results with live market price data fetched from the UN Comtrade API. A RAG pipeline using FAISS vector search and the all-MiniLM-L6-v2 sentence embedding model retrieves relevant regulatory and market context, which is passed to the Groq-hosted LLama LLM to generate country-specific export recommendations.

The third component is the User Interface and Reporting Layer, a Streamlit web application accessible from any browser on the local network. It provides real-time inspection feedback, export advisory dashboards, an interactive chat interface for export queries, and automated PDF report generation containing inspection images, quality metrics, and export recommendations.

1.5. Project Scope

ExportEdge AI addresses the following in-scope functionalities: video-based mango inspection from either uploaded video files or a live RTSP camera feed, disease classification and segmentation, surface defect analysis, UN Comtrade price-based export advisory, RAG-powered chat for export regulation queries, and PDF inspection report generation. The system is designed for single-device edge deployment on a Raspberry Pi 5 with headless operation and auto-start capability via system.

1.6. Stakeholders

The primary stakeholders of ExportEdge AI are mango exporters and packhouse operators who require fast, consistent quality grading without specialist technical knowledge. Secondary stakeholders include agricultural inspection bodies such as PASSCO and the Plant Protection Department, who may use inspection reports for compliance verification. System administrators and developers constitute the technical stakeholder class, responsible for model updates, maintenance, and system configuration.

1.7. Technology Stack

The tech stack of ExportEdge AI consists of following technologies:

- Python>=3.12.0
- Streamlit>=1.28.0
- Opencv>=4.8.0
- Numpy>=1.24.0
- Tensorflow>=2.13.0
- Pytorch>=2.0.0
- Langchain>=0.1.0
- Faiss-cpu>=1.7.4
- Jinja2>=3.1.0
- Sentence-transformers>=2.2.0
- Llama
- Xhtml2pdf
- Raspberry Pi5 (4GB)
- UN Comtrade API

2. Software Requirements Specifications

2.1. Requirements Elicitation Process

The requirements for ExportEdge AI were elicited through a combination of stakeholder analysis, domain research, and iterative prototyping. Initial requirements were gathered through discussions with the project supervisor Dr. Shahzad Younis, focusing on the core problem of automated mango quality inspection for export. Domain research into Pakistan's mango export industry, UN Comtrade pricing data, and existing agricultural AI literature informed the functional scope. Requirements were further refined iteratively as the system was developed and tested, with changes documented in the SRS revision history.

The elicitation process identified three primary user classes: mango exporters who require actionable quality and market intelligence, system administrators responsible for deployment and maintenance.

2.2. Use Case Diagram

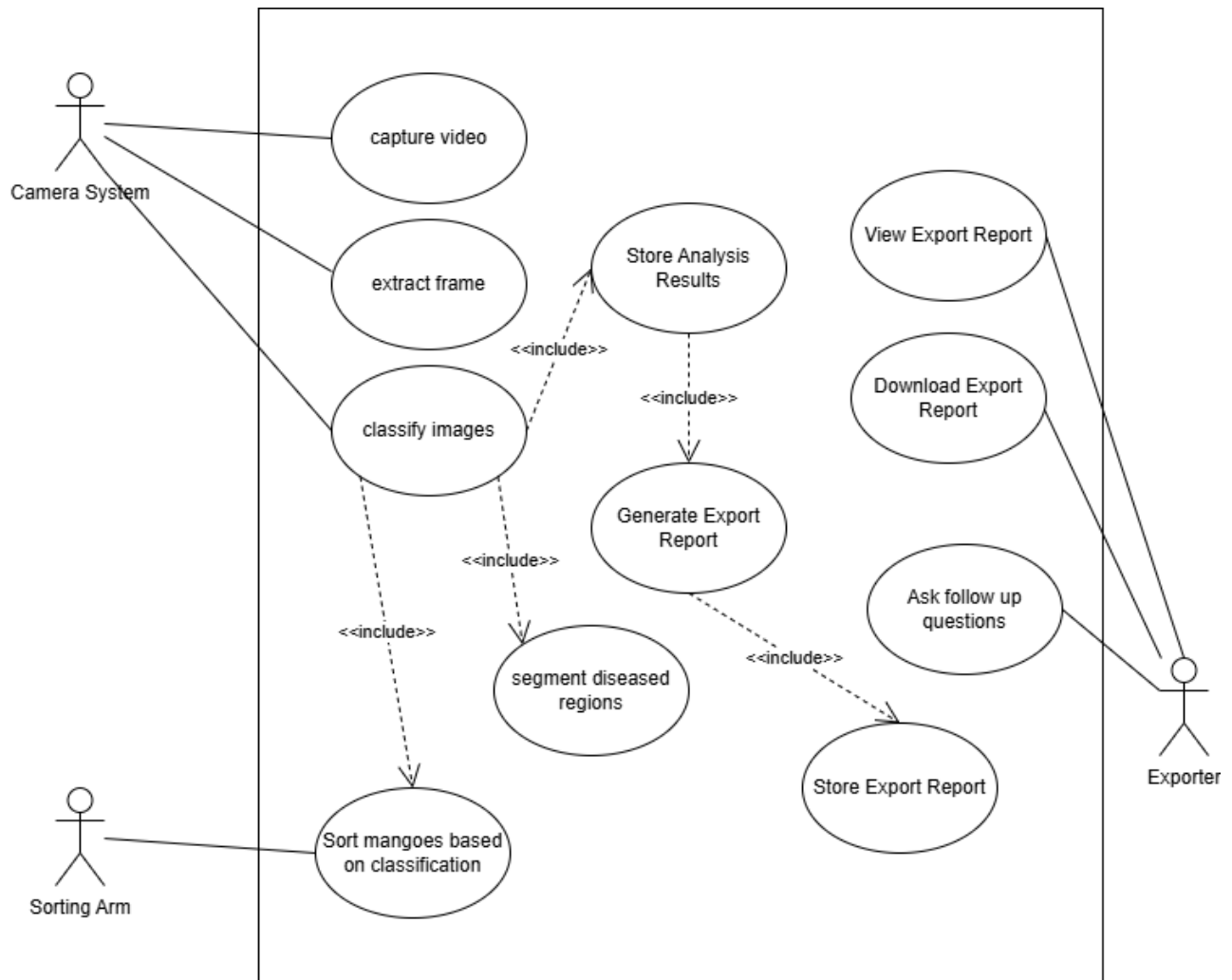


Figure 2.1: Use Case Diagram for ExportEdge

2.2.1. User Scenarios

Example Scenario 1: Mango Exporter Inspecting a Batch for Export

- **User Type:** Mango exporter at a packhouse preparing a batch for international shipment.
- **User Interaction:**
 - **Input:** The exporter opens the ExportEdge AI web interface from their laptop browser, selects Video File as the input source, and uploads a pre-recorded conveyor belt video of the mango batch.
 - **Action:** The exporter clicks Start Inspection and the system begins processing the video automatically.
- **System Response:**
 - The system detects motion in the video, extracts the peak-motion frame, and runs the TFLite MobileNetV2 classification model. The result is displayed as Healthy with 94% confidence.

- The DeepLabV3 segmentation model runs and overlays a minimal disease mask on the mango image. The defect analysis module assigns a surface quality score of 92/100, color uniformity score of 88/100, and Export Grade A.
- The UN Comtrade API returns live price data showing UAE at \$1.8/kg, UK at \$2.1/kg, and Germany at \$2.3/kg as the top recommended markets.
- The Groq LLama LLM generates a natural language recommendation: "This Grade A mango is suitable for premium export markets including Germany and the UK. Ensure phytosanitary certification and cold-chain compliance before shipment."
- **Next Steps:**
 - The exporter clicks Generate PDF Report and downloads the formal inspection document.
 - The report is automatically embedded into the FAISS vector store for follow-up queries.

Example Scenario 2: Exporter Querying Export Regulations via Chat

- **User Type:** Mango exporter unfamiliar with EU phytosanitary requirements.
- **User Interaction:**
 - **Input:** After completing an inspection, the exporter navigates to the Chat page and types: "What are the phytosanitary requirements for exporting mangoes to the EU?"
 - **Action:** The exporter submits the query via the chat interface.
- **System Response:**
 - The RAG pipeline retrieves relevant context from the FAISS vector store including the current session's inspection report and regulatory SOP documents covering EU export standards.
 - The Groq LLaMA LLM generates a grounded response: "For EU export, mangoes must comply with EU Regulation 543/2011. A phytosanitary certificate issued by the National Plant Protection Organization is mandatory. Hot water treatment at 48°C for 60 minutes is required to meet pest-free standards. Your current batch is Grade A which qualifies for EU premium markets."
 - Source citations from the regulatory documents are displayed below the response.
- **Next Steps:**
 - The exporter follows up with a second question: "Which documents do I need for Germany specifically?"
 - The system retrieves Germany-specific regulatory context and generates a targeted response listing required documentation.
 - The exporter uses the information to prepare the shipment documentation with confidence.

2.3. Functional Requirements

Identifier	FR-1
Title	Video Input Upload
Requirement	The system shall accept video input from an uploaded MP4, AVI, MOV, or MKV file up to 200MB in size. The user shall be able to upload the file via the web interface before starting inspection.
Source	Developer
Rationale	Allows users to inspect mangoes from pre-recorded conveyor belt footage when a live camera is not available.
Business Rule (if required)	Only MP4, AVI, MOV, and MKV formats are supported.
Dependencies	None
Priority	High

Identifier	FR-2
Title	Live Camera Feed input
Requirement	The system shall accept live video input from a configured RTSP camera feed. The user shall be able to select Camera Feed as the input source and the system shall connect to the configured camera URL.
Source	Developer
Rationale	Enables real-time mango inspection on the conveyor belt using the Dahua IP camera connected to the Raspberry Pi

	5.
Business Rule (if required)	N/A
Dependencies	None
Priority	High

Identifier	FR-3
Title	Motion Detection and Frame Extraction
Requirement	The system shall continuously monitor the video stream for motion. Upon detecting motion above a configured threshold, the system shall extract the peak-motion frame representing the mango for analysis.
Source	Developer
Rationale	Ensures that only frames containing a mango are passed to the AI pipeline, preventing unnecessary inference on empty conveyor frames.
Business Rule (if required)	N/A
Dependencies	FR-1, FR-2
Priority	High

Identifier	FR-4
Title	Image Processing
Requirement	The system shall preprocess each extracted mango frame before passing it to the classification model. Preprocessing shall include resizing to 150×150 pixels, denoising, colour correction, unsharp masking, contrast enhancement, and ImageNet normalization.
Source	Developer
Rationale	Ensures consistent model input quality and improves classification accuracy by standardizing image characteristics across different lighting and camera conditions.
Business Rule (if required)	N/A
Dependencies	FR-3
Priority	High

Identifier	FR-5
Title	Mango Disease Classification
Requirement	The system shall classify each preprocessed mango frame as Healthy or Diseased using a TFLite MobileNetV2 model deployed on the Raspberry Pi 5. The system shall display the classification label and confidence percentage to the user.
Source	Developer
Rationale	Disease classification is the primary quality gate of the system. Accurate classification determines export

	eligibility and triggers downstream processing..
Business Rule (if required)	N/A
Dependencies	N/A
Priority	High

Identifier	FR-6
Title	Disease Segmentation
Requirement	The system shall run a TFLite DeepLabV3 segmentation model on each inspected mango frame. The system shall generate a pixel-level disease mask, overlay it on the original image using a red highlight, and calculate the disease area as a percentage of total mango surface.
Source	Developer
Rationale	Segmentation provides a visual and quantitative measure of disease extent, enabling exporters to make informed decisions about the severity of infection beyond a binary classification result.
Business Rule (if required)	N/A
Dependencies	FR-5
Priority	High

Identifier	FR-7
Title	Defect Analysis
Requirement	The system shall compute a surface quality score (0–100), a color uniformity score (0–100), and assign an export grade (Grade A, B, or C) for each inspected mango. The system shall display an impact level (minimal, moderate, or significant) based on the computed scores.
Source	Developer
Rationale	Surface defect analysis provides a more comprehensive quality assessment beyond disease detection, supporting accurate export grading aligned with international standards.
Business Rule (if required)	N/A
Dependencies	N/A
Priority	High

Identifier	FR-8
Title	Export Price Advisory
Requirement	The system shall fetch live mango export price data from the UN Comtrade API for a predefined set of target countries. The system shall display a table of recommended export destinations with price per kg (USD) and data period. Diseased mangoes shall be flagged as ineligible for export.
Source	Developer

Rationale	Providing real-time market prices enables exporters to direct their produce to the most profitable destination markets, directly addressing the business opportunity of maximizing export revenue.
Business Rule (if required)	Export recommendations shall only be generated for mangoes classified as Healthy.
Dependencies	FR-5, FR-7
Priority	High

Identifier	FR-9
Title	RAG-Based Export Intelligence
Requirement	The system shall use a FAISS vector store with all-MiniLM-L6-v2 sentence embeddings to retrieve relevant export regulatory and market context. The retrieved context along with inspection results shall be passed to a Groq-hosted LLaMA LLM to generate a natural language export recommendation displayed to the user.
Source	Developer
Rationale	Pure price data is insufficient for export decisions. Regulatory compliance, phytosanitary requirements, and market demand trends require contextual reasoning that an LLM with RAG can provide.
Business Rule (if required)	LLM responses shall be grounded only in retrieved context to prevent hallucination.
Dependencies	N/A
Priority	High

Identifier	FR-10
Title	PDF Report Generation
Requirement	The system shall generate a downloadable PDF inspection report upon user request. The report shall include the inspection timestamp, video source, original ROI image, segmentation overlay image, classification result, quality metrics, export grade, UN Comtrade price table, and LLM export recommendation.
Source	Developer
Rationale	Exporters and inspection bodies require formal documentation of inspection results for compliance verification, record-keeping, and dispute resolution at destination ports.
Business Rule (if required)	LLM responses shall be grounded only in retrieved context to prevent hallucination.
Dependencies	FR-5, FR-6, FR-7, FR-8, FR-9
Priority	High

Identifier	FR-11
Title	Automatic Report Embedding into Vector Store
Requirement	Upon generation of an inspection report, the system shall automatically convert the report content into vector embeddings using the all-MiniLM-L6-v2 sentence embedding model and store them in the FAISS vector database. The stored embeddings shall be immediately

	available for retrieval so that the user can ask follow-up questions about the current inspection report via the chat interface.
Source	Developer
Rationale	Automatically indexing the inspection report into the vector store enables the RAG pipeline to retrieve inspection-specific context when the user asks follow-up questions such as "Why was this mango graded B?" or "Which country should I export this batch to?" without the user needing to manually upload or reference the report.
Business Rule (if required)	Report embeddings shall be generated and stored before the chat interface becomes available for that inspection session.
Dependencies	FR-10
Priority	High

Identifier	FR-12
Title	System Auto-Start on Boot
Requirement	The system shall automatically start the Streamlit web application on Raspberry Pi boot via a systemd service. The service shall restart automatically on failure. The application shall be accessible from any browser on the local network at port 8501 without requiring manual intervention.
Source	Developer
Rationale	In a packhouse environment, the system must be ready for use immediately upon powering on the Raspberry Pi without requiring technical knowledge to start the

	application.
Business Rule (if required)	N/A
Dependencies	N/A
Priority	High

2.4. Non-Functional Requirements

2.4.1. Performance Requirements

- **Classification Inference Time:** The system shall complete mango disease classification inference within 2 seconds per frame on the Raspberry Pi 5 4GB edge device.
- **Segmentation Inference Time:** The system shall complete disease segmentation inference and overlay generation within 2 seconds per frame on the Raspberry Pi 5 4GB edge device.
- **Full Pipeline Throughput:** The complete inspection pipeline including classification, segmentation, and defect analysis shall complete within 10 seconds per mango inspection event.
- **Export Advisory Response Time:** The system shall fetch UN Comtrade price data and generate the LLM-based export recommendation within 10 seconds of inspection completion.
- **Chat Response Time:** The RAG-based chat interface shall retrieve context and generate a natural language response to user queries within 5–10 seconds.
- **Report Generation Time:** The system shall generate and make available the PDF inspection report within 5 seconds of the user clicking the Generate Report button.
- **UI Responsiveness:** The web interface shall respond to user interactions such as button clicks, page navigation, and video upload within 2 seconds under normal operating conditions.

2.4.2. Safety Requirements

- **Model Failure Handling:** If the classification or segmentation model fails to process a frame, the system shall display a descriptive error message to the user and continue monitoring the video stream without crashing.
- **API Failure Handling:** If the UN Comtrade API or Groq LLM API is unavailable, the system shall display an appropriate error message and allow the user to retry without affecting other system functionalities.
- **Data Integrity:** All inspection results, generated reports, and vector embeddings shall be stored and processed securely to prevent accidental loss or corruption during operation.

- **Failure Recovery:** The system shall automatically restart within 5 seconds of a service failure via the systemd auto-restart mechanism configured with Restart=on-failure and RestartSec=5.
- **Safe LLM Responses:** Guard rails shall be implemented in the system so that LLM responses are relevant to mango export and do not generate harmful, misleading, or off-topic content.
- **Error Messaging:** Clear and descriptive error messages shall be displayed to users for all failure scenarios including video upload errors, model loading failures, API timeouts, and report generation failures, guiding the user toward resolution.

2.4.3. Security Requirements

- **API Key Protection:** API keys for the Groq LLM service and UN Comtrade data API shall be stored in a .env file and shall never be hardcoded in source code or exposed in the user interface.
- **Model File Protection:** Trained TFLite model files shall be read-only during deployment to prevent accidental overwriting or tampering.
- **Local Network Access Control:** The Streamlit web application shall only be accessible within the local network. No external internet exposure of the application shall be configured by default.
- **Data Privacy:** Mango inspection images and generated reports shall be processed and stored locally on the Raspberry Pi 5 and shall not be transmitted to third-party services, except for LLM query text sent to the Groq API.

3. Software Design Specifications

3.1. Architectural Style

The ExportEdge AI system employs an Edge-Optimized Layered Architecture, organized into four distinct layers that enforce a strict sequential data flow from hardware input to user presentation. This design promotes scalability, simplifies maintenance, and ensures low-latency performance despite deploying all AI components locally on the resource-constrained Raspberry Pi 5 edge device. Below is an in-depth breakdown of the system's key layers and their interactions.

3.1.1. Data Acquisition and Persistence Layer

- **Purpose:** Manages the physical hardware interface, captures video input from the uploaded file or RTSP camera feed, and performs initial frame processing to optimize input for the AI pipeline. This layer also manages all data persistence including temporary image storage and structured report storage.
- **Technology:** Python with OpenCV 4.13 for video capture, HSV-based motion detection, ROI extraction, morphological cleanup, and image preprocessing. File system for temporary image storage and SQLite for structured report storage.
- **Interaction:** Receives raw video input, applies motion detection to identify mango-containing frames, extracts and preprocesses the Region of Interest (ROI), and passes the processed frame to the AI Analysis Layer.

3.1.2. AI Analysis Layer

- **Purpose:** Executes the core computer vision tasks, disease classification, disease segmentation, and surface defect analysis, at the edge without requiring cloud connectivity.
- **Technology:** TFLite Runtime 2.14.0 on Python 3.11.9 for efficient inference of the MobileNetV2 classification model (8.5MB, float32) and the DeepLabV3 segmentation model (21MB, float16). OpenCV handles color space conversion, overlay generation, and defect metric computation.
- **Interaction:** Receives a preprocessed frame from the Data Acquisition Layer. Executes classification first, followed by segmentation for all mangoes regardless of classification result. Computes surface quality score, color uniformity score, and export grade from the combined outputs.

3.1.3. Intelligence Layer

- **Purpose:** Serves as the application and business logic layer, orchestrating the system workflow, executing the LLM, integrating live market data, and generating high-value export reports.
- **Technology:** Python application using LangChain for RAG orchestration, FAISS for vector similarity search, all-MiniLM-L6-v2 (deployed locally on Pi) for sentence embeddings, and the Groq API (OpenAI-compatible) for LLaMA LLM inference. UN Comtrade API for live mango export price retrieval.
- **Interaction:** After inspection completes, fetches UN Comtrade price data, retrieves relevant regulatory context from the FAISS vector store, and passes the combined context to the Groq LLM to generate export recommendations.

Automatically embeds generated reports into the FAISS vector store for follow-up chat queries.

3.1.4. Frontend Layer

- **Purpose:** Provides the user interface and acts as the presentation layer, delivering real-time inspection feedback, export advisory dashboards, and PDF reports to the exporter through a browser-accessible web application.
- **Technology:** Streamlit web framework accessible from any browser on the local network at port 8501. Custom CSS for professional styling. xhtml2pdf for PDF report generation.
- **Interaction:** Renders the live video stream, displays classification results with confidence scores, segmentation overlays with disease masks, defect analysis metrics, UN Comtrade price tables, LLM recommendations, and hosts the RAG-based chat interface.

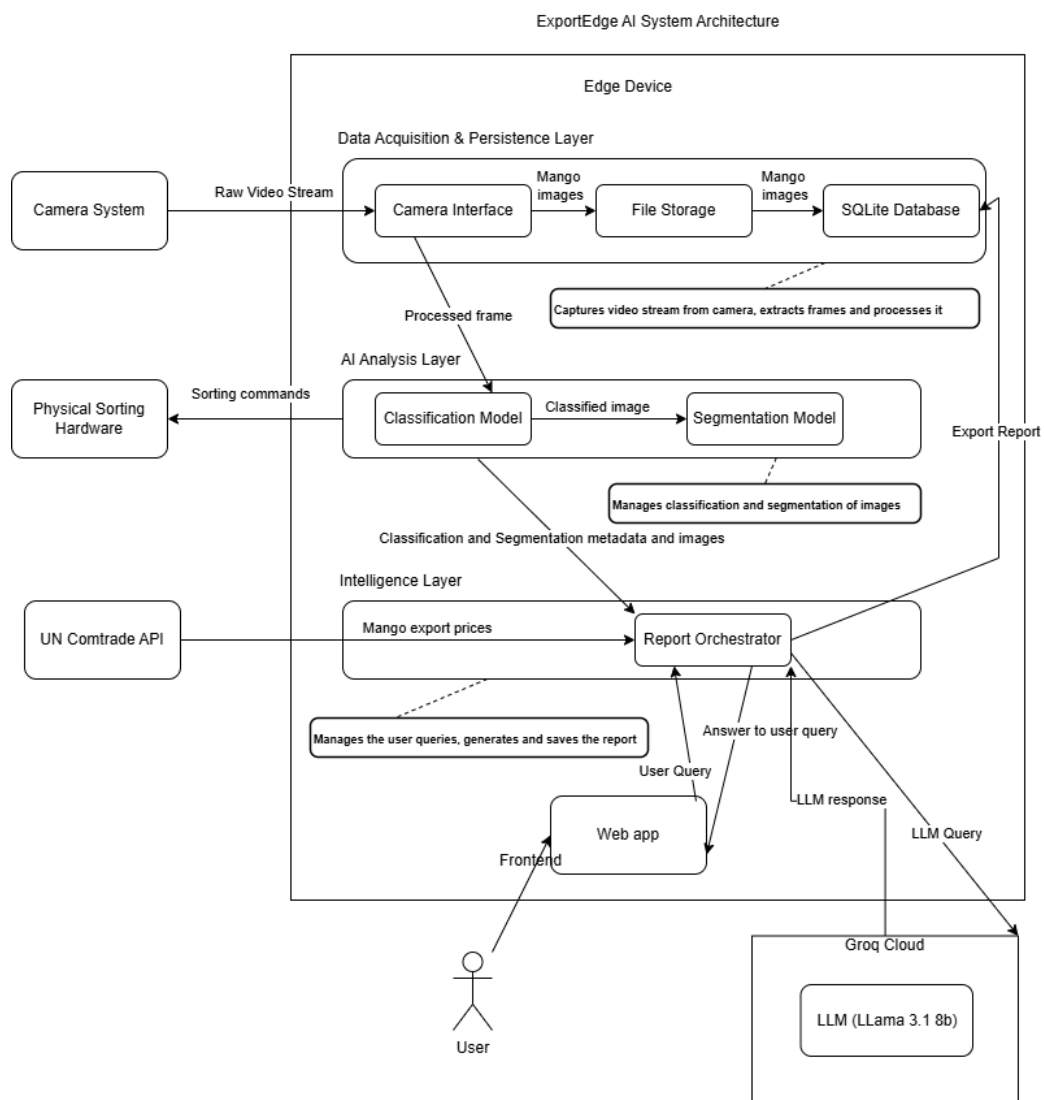


Figure 3.1: Architecture Diagram for ExportEdge

3.2. Selection of Design Methodology

The project employs a modular object-oriented design methodology, combined with an Incremental and Iterative Software Process Model. This combination was selected to effectively manage the complexity of a multi-stage AI-driven system involving image-based disease detection, segmentation, regulatory compliance analysis, and export decision-making.

3.2.1. Simplicity

The object-oriented modular design simplifies system development by dividing the overall workflow into clearly defined components, mango image classification, disease segmentation, surface defect analysis, export price advisory, RAG-based intelligence, and PDF report generation. Each component is implemented as a self-contained module, making the system easier to understand, develop, and debug.

3.2.2. Modularity

Each module performs a specific, independent function, for example, the classification module, segmentation module, and export advisory module , allowing focused development and testing of each component. This modularity simplifies future enhancements or replacements without affecting the entire system.

3.2.3. Scalability

The modular design enables the system to scale efficiently. New disease classes, improved TFLite models, or additional fruit types can be incorporated by extending existing classes or replacing model files without requiring major changes to the core system architecture.

3.2.4. Flexibility

The incremental and iterative model ensures each module can be refined and adjusted to accommodate new requirements or improve performance. For example, the segmentation model was iteratively debugged to resolve output shape mismatches and numpy compatibility issues across development iterations.

3.3. Data Representation Diagram

The Entity Relationship Diagram below illustrates the persistent data structure of ExportEdge AI. The system uses a decision-centric data design approach where only finalized export intelligence is persistently stored. Intermediate data such as raw mango images and model inference outputs are treated as temporary artifacts and discarded after report generation.

The two primary database entities are ExportReport and ExportReportItem. One ExportReport can contain multiple ExportReportItem records, representing a one-to-many relationship. Each ExportReportItem corresponds to a single mango inspection

result and contains all information required for visualization, decision-making, and report rendering.

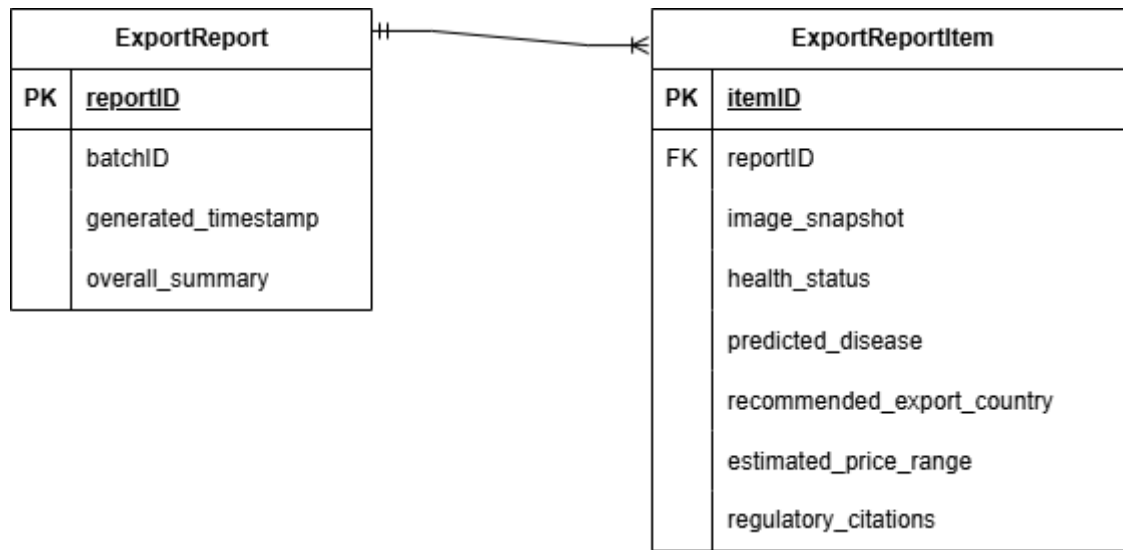


Figure 3.2: ERD for ExportEdge

3.4. Process Flow

The process flow diagram below illustrates the complete sequential pipeline of ExportEdge AI from video input to PDF report generation. The process begins with video capture and motion detection in the Data Acquisition Layer. Upon detecting a mango, the peak-motion frame is extracted and preprocessed. The AI Analysis Layer runs classification followed by segmentation. The Intelligence Layer then fetches UN Comtrade price data, retrieves regulatory context via RAG, and generates the LLM export recommendation. The generated report is embedded into the FAISS vector store and made available for user download and follow-up chat queries.

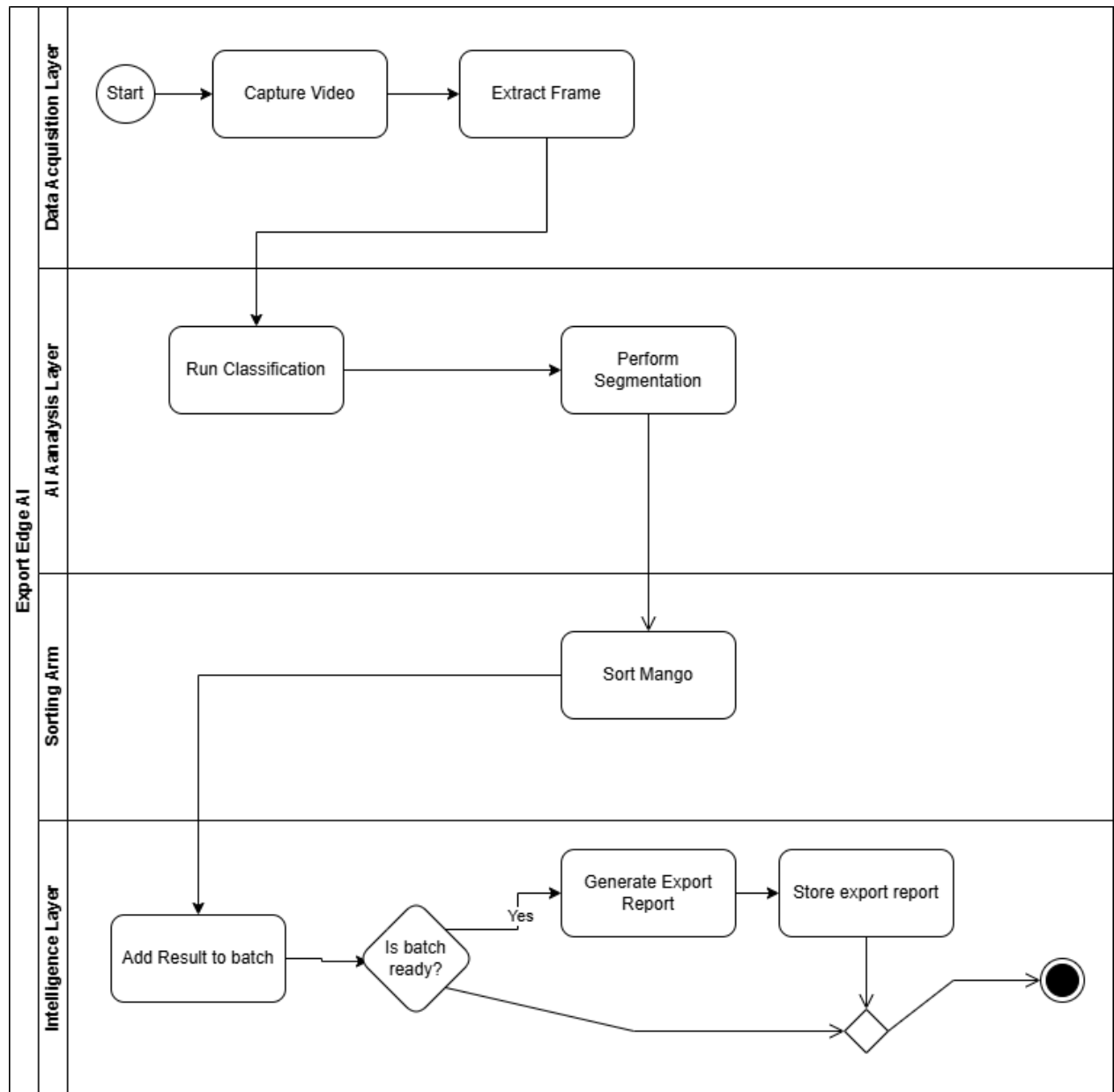


Figure 3.3: Process Flow Model for ExportEdge

3.5. Design Models

3.5.1. Data Flow Diagrams

3.5.1.1. DFD Level 0: Context Diagram

- **Purpose:** Defines the system boundary for the export grading and market analysis platform.
- **Concept:** It treats the entire ExportEdge AI application as a single "Black Box" process (P0).
- **Flow:** It shows four external entities interacting with the core process: The Camera and Market Data provide sensory and financial inputs, while the Exporter acts as the primary user sending queries and receiving responses. The Sorting Arm receives automated physical commands based on the system's processing.

- **Outcome:** It provides a high-level view of how raw video and price data are converted into actionable sorting instructions and exporter feedback while hiding all internal RAG logic or database structures.

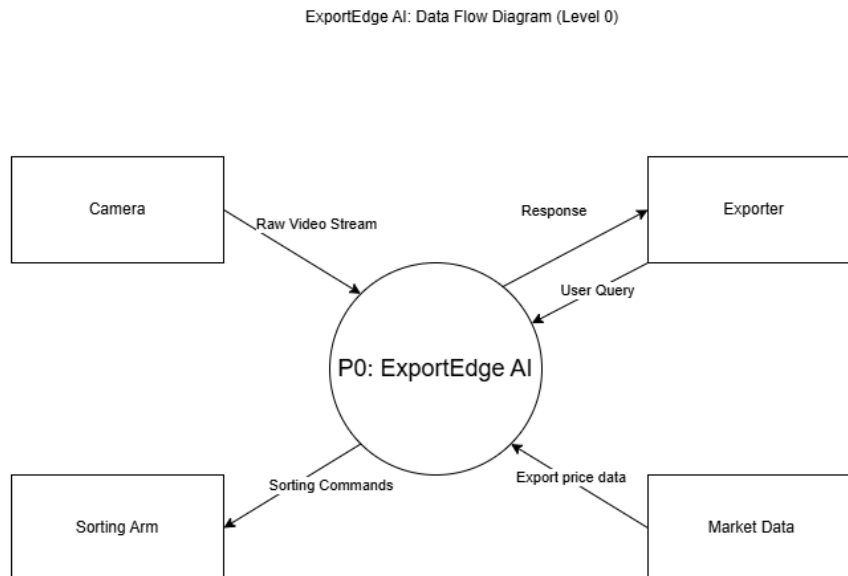


Figure 3.4: Context Diagram for ExportEdge

3.5.1.2. DFD Level 0: High Level System Decomposition

- **Purpose:** Breaks the "Black Box" into four main functional engines to manage mango processing and exporter interactions.
- **Processes:**
 1. **Capture & Process Mango Image Data (P1):** Converts the raw video stream from the camera into processed image data and stores it temporarily.
 2. **Analyze Mango Quality (P2):** The computer vision engine that performs classification and segmentation to determine fruit quality and control the sorting arm.
 3. **Orchestrate and Generate Report (P3):** Combines quality metadata with market price data to create comprehensive export reports.
 4. **Present Report & Answer Queries (P4):** The interaction layer that delivers reports to the exporter and provides LLM-based answers to user queries.
- **Data Stores:** Reveals the three internal databases:
 - **D1: Temporary Image Storage** for holding raw visual data.
 - **D2: Export Report Database** for storing generated analytical summaries.

- **D3: Vector Knowledge Store** which provides context for the RAG-based query system.

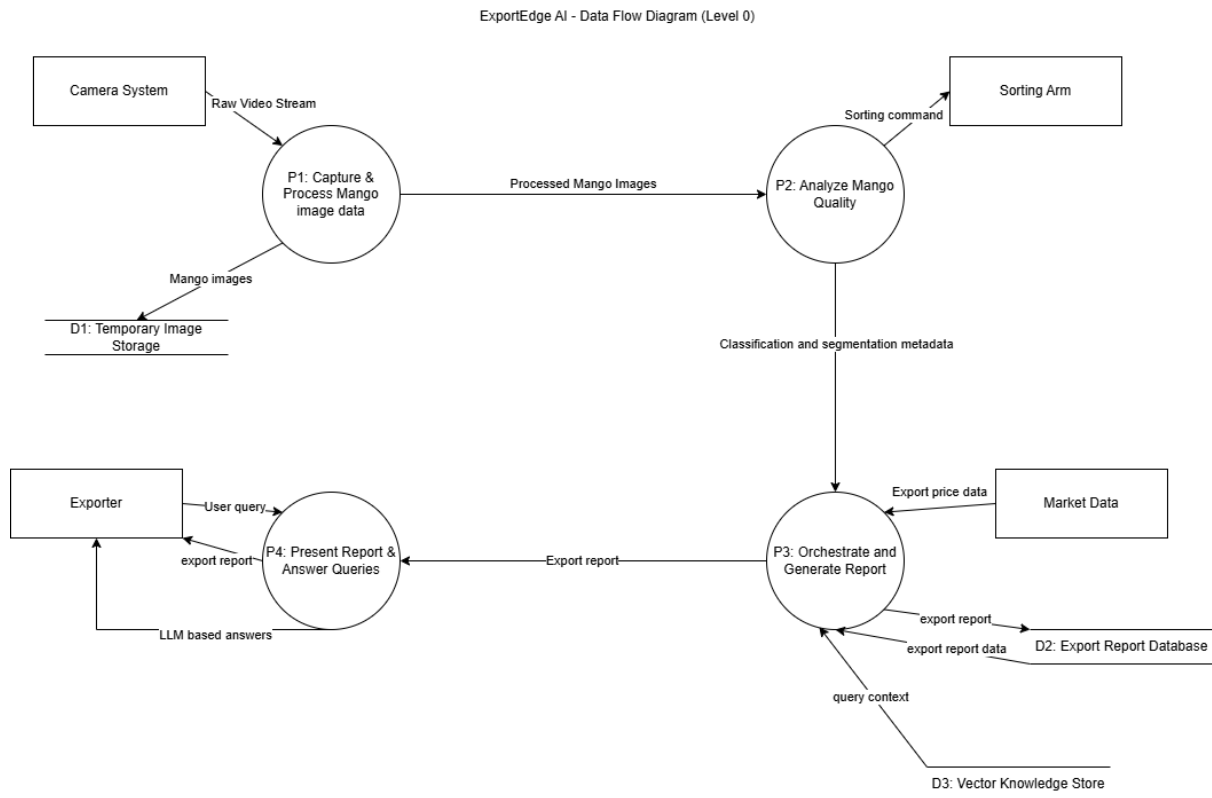


Figure 3.5: DFD Level0 for ExportEdge

3.5.1.3. DFD Level 1: Analyze Mango Quality

- **Purpose:** Decomposes the quality analysis process into three functional sub-processes to detail the computer vision pipeline logic.
- **Logic:**
 - **Classify Mango Image (2.1)** receives processed frames and retrieves image data from D1: Temporary Image Storage to perform grading.
 - If a disease is identified, the system triggers Segment Defects (2.2) to calculate the specific area of infection.
 - Simultaneously, classification results flow to Issue Sorting Command (2.3).
- **Purpose:** Decomposes the quality analysis process into three functional sub-processes to detail the computer vision pipeline logic.

ExportEdge AI - Data Flow Diagram (Level 1)

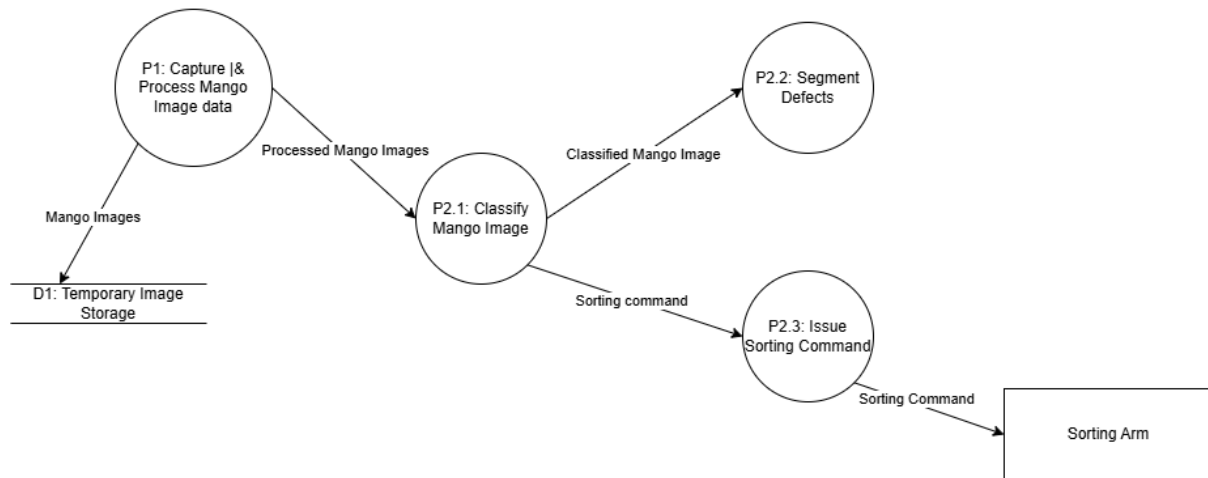


Figure 3.6: DFD Level1 for Analyze Mango Quality

3.5.1.4. DFD Level 1: Orchestrate and Generate Report

- **Purpose:** Decomposes the intelligence layer into sub-processes to manage data batching, market analysis, and the RAG-based LLM service.
- **Logic:**
 - **Manage Batch (3.1)** collects metadata from the quality analysis process until a batch is ready for processing.
 - **Market Analysis (3.2)** fuses this batch data with external Export price data to identify profitable destinations.
 - **LLM & RAG Service (3.3)** takes the structured data and retrieves context from D3: Vector knowledge store to produce a natural language report.
- **Outcome:** The Save Report (3.4) process ensures the final document is persisted into D2: Export Report Database for future user retrieval.

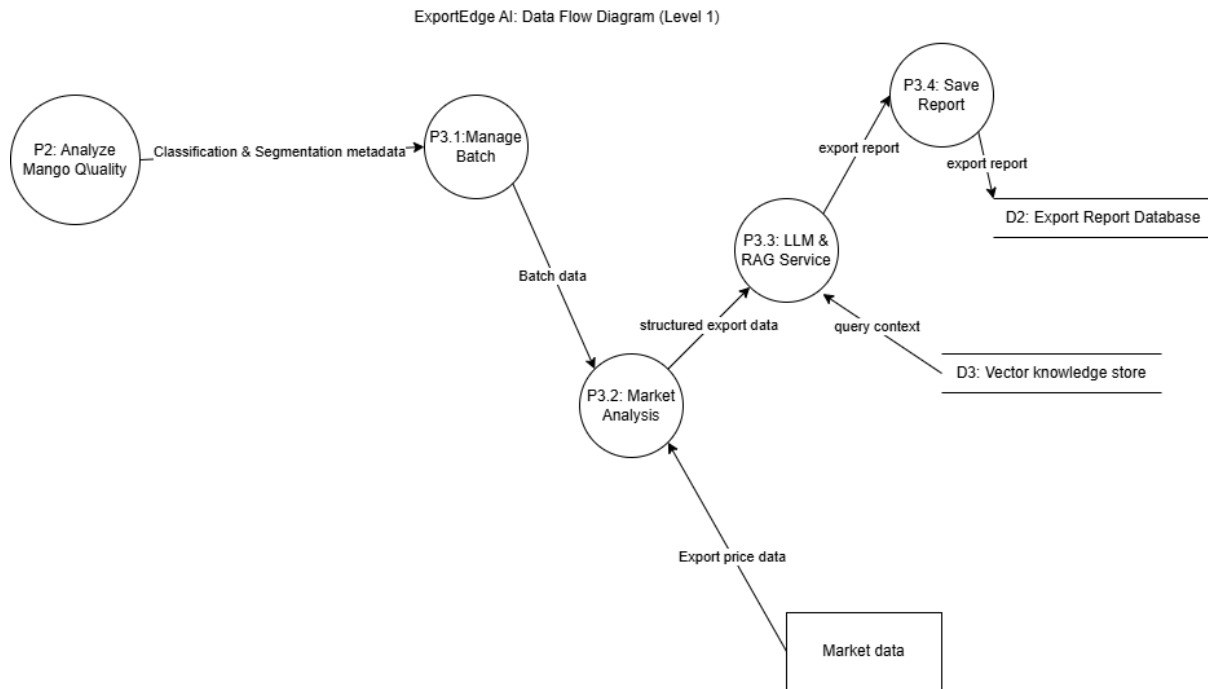


Figure 3.7: DFD Level1 for Orchestrate and Generate Report

3.5.2. Sequence Diagrams

3.5.2.1. Sequence Diagram for Automated Mango Processing Pipeline

Sequence of Events:

1. The camera system triggers the data acquisition layer to capture the live video stream.
2. The data acquisition layer extracts and pre-processes the frame extracted and sends the processed image to the AI analysis layer.
3. The AI analysis layer executes the classification process on the image.
4. Based on the classification result, the AI analysis layer immediately sends the command to sorting arm to sort the mango to respective bin (real-time).
5. The AI analysis layer then executes segmentation to recognize defected regions in the mango.
6. The analysis results (classification and segmentation metadata) are then sent to the Intelligence Layer.
7. The intelligence layer executes the generate export report process to generate the export report using LLM and market data.
8. Finally, the intelligence layer stores the generated export report in the database.

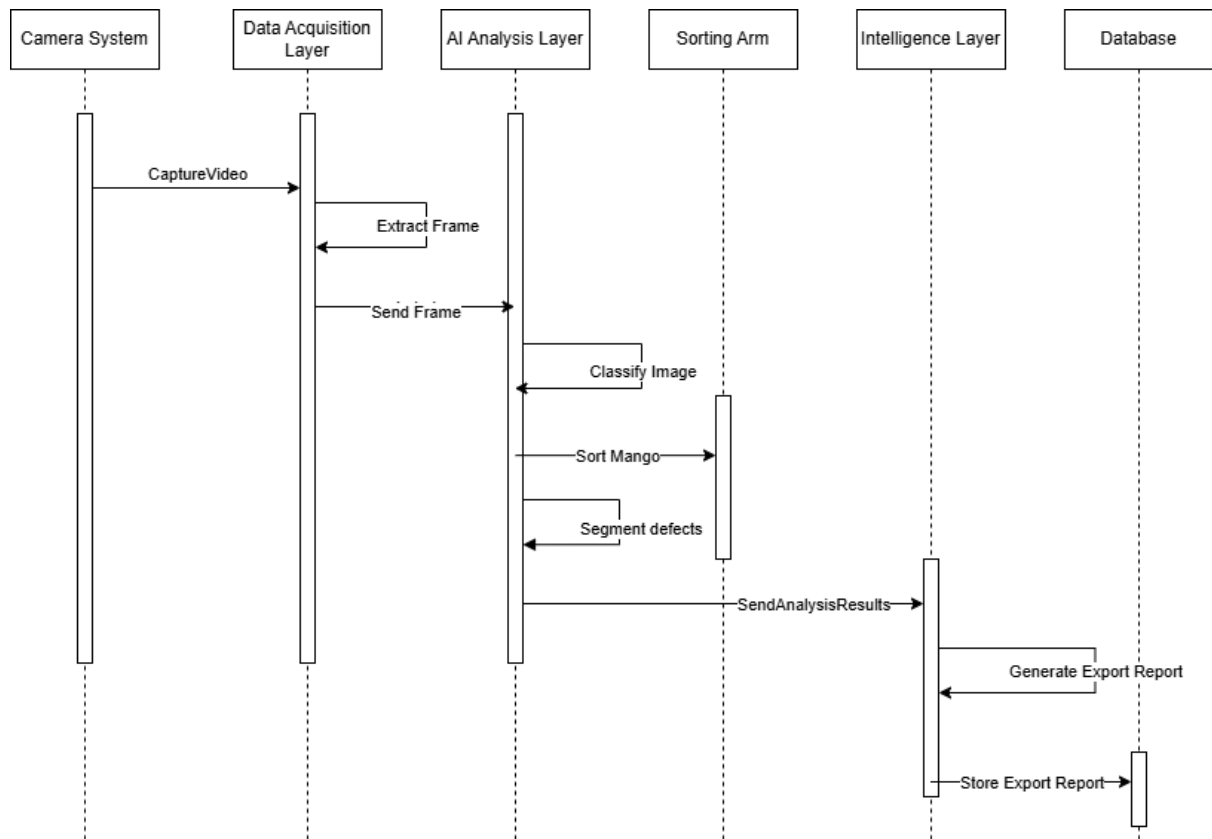


Figure 3.8: Sequence Diagram for Automated Mango Processing Pipeline

3.5.2.2. Sequence Diagram for Frontend Report Retrieval

Sequence of Events:

1. The user initiates the process by sending a request to view the export report.
2. The frontend web app sends a message to the intelligence layer to get the export report.
3. The intelligence layer executes the `getExportReport()` method to send a request to the database.
4. The database retrieves the required data and returns the data back to the intelligence layer.
5. The intelligence layer creates the report in the required format and then sends the final export report back to the frontend.
6. The frontend renders and displays the export report to the user.

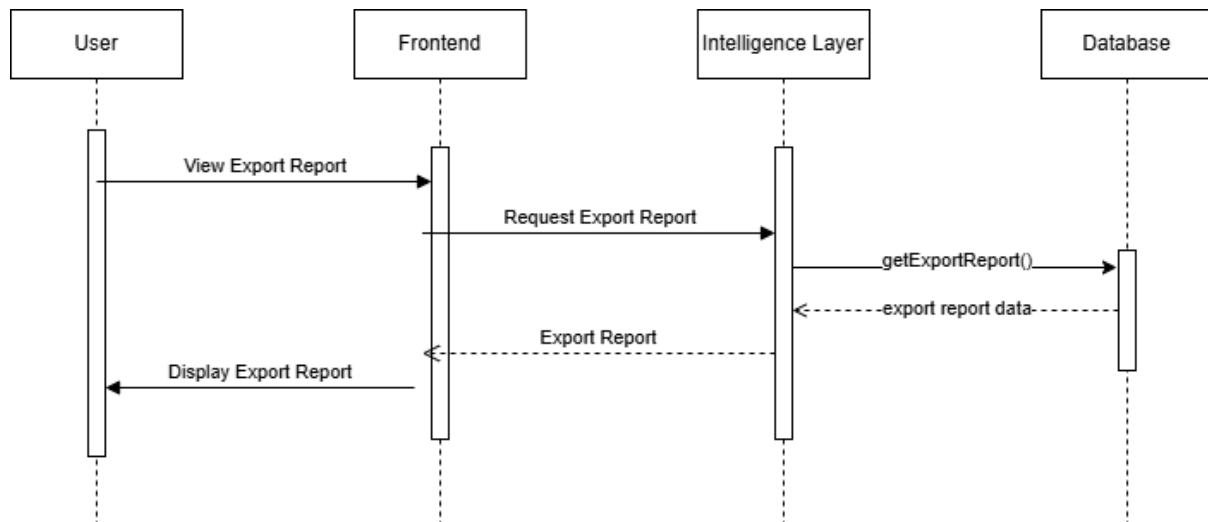


Figure 3.9: Sequence Diagram for Frontend Report Retrieval

3.5.2.3. Sequence Diagram for User Query and LLM Response

Sequence of Events:

1. The user initiates the process by submitting a query.
2. The frontend forwards the query to the intelligence layer.
3. The intelligence layer processes the query by initiating the RAG process and generating the final answer using the LLM.
4. The intelligence layer sends the generated answer back to the frontend.
5. The frontend then renders and shows the answer to the user.

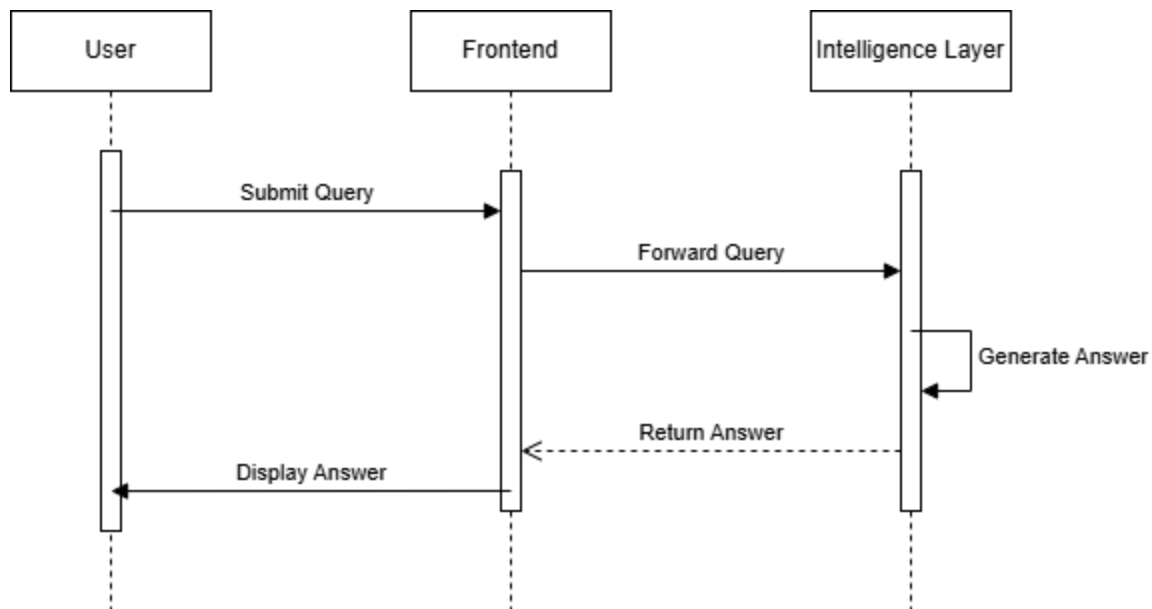


Figure 3.10: Sequence Diagram for User Query and LLM Response

3.5.3. State Transition Diagrams

3.5.3.1. *Packhouse Sorting Controller State Machine*

This State Machine Diagram models the dynamic, real-time lifecycle of a mango as it is processed by the edge-deployed Computer Vision (CV) pipeline. It defines the high-speed operational states required to ensure accurate grading and physical sorting on the conveyor belt.

State and Transition Description:

- **Idle:** The default state where the system waits for a mango to enter the camera's field of view. A Motion detected event triggers the transition to processing.
- **Acquiring:** The camera interface captures the specific frame for analysis. Upon Frame captured, the system moves to AI inference.
- **Classifying:** The Mobile Net model runs to determine the mango's health and grade. Once classification completed, the system proceeds to detailed surface analysis.
- **Segmenting:** The DeepLab v3 model segments the mango surface to identify irregularities or defects. When the mask generated event occurs, the system moves to physical action.
- **Sorting:** The system sends a signal to the sorting arm. Once the Sorting Command sent event is complete, the controller returns to the Idle state for the next item.

Packhouse Sorting Controller State Machine Diagram

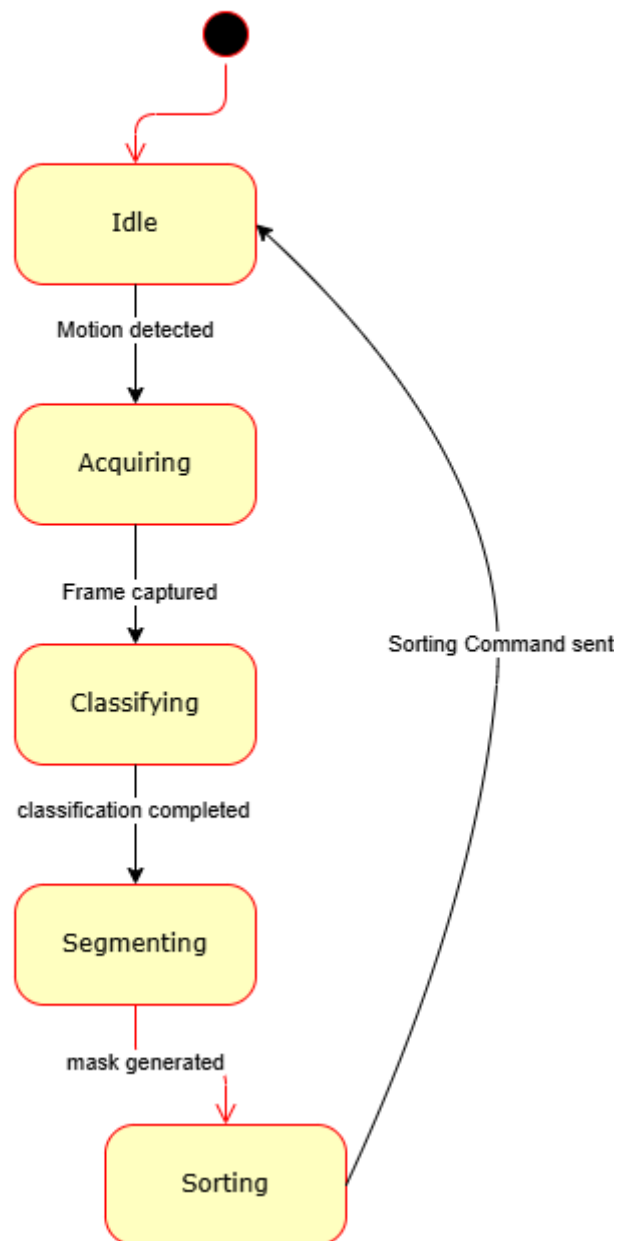


Figure 3.11: State Machine for Packhouse sorting controller

3.5.3.2. Intelligence Orchestrator State Machine

This diagram models the logical states of the system as it orchestrates batch management, integrates market data, and utilizes the RAG framework to generate narrative export intelligence.

State and Transition Description:

- **Buffering:** The initial state where the system stays as it receives "Classification & Segmentation metadata" from the analysis layer. It remains here as long as `isBatchReady == False`.
- **Market Analyzing:** Triggered when `isBatchReady == True`. The system calculates optimal pricing and destination recommendations based on aggregated batch data.
- **Retrieving Context:** Upon the Market data fetched event, the orchestrator initiates the RAG process, querying the Vector Knowledge Store (D3) for regulatory and market compliance documents.
- **Generating Narrative:** Entered when context retrieved. The system is in this state while the LLM (e.g., Llama-3.1-8B) is actively processing the retrieved context and batch data to produce the final narrative report.
- **Finalizing:** Triggered by response generation complete. The system completes the lifecycle by saving the report to the Export Report Database and presenting it to the exporter. Upon Save Successful, the orchestrator returns to the Buffering state.

Intelligence Orchestrator State Transition Diagram

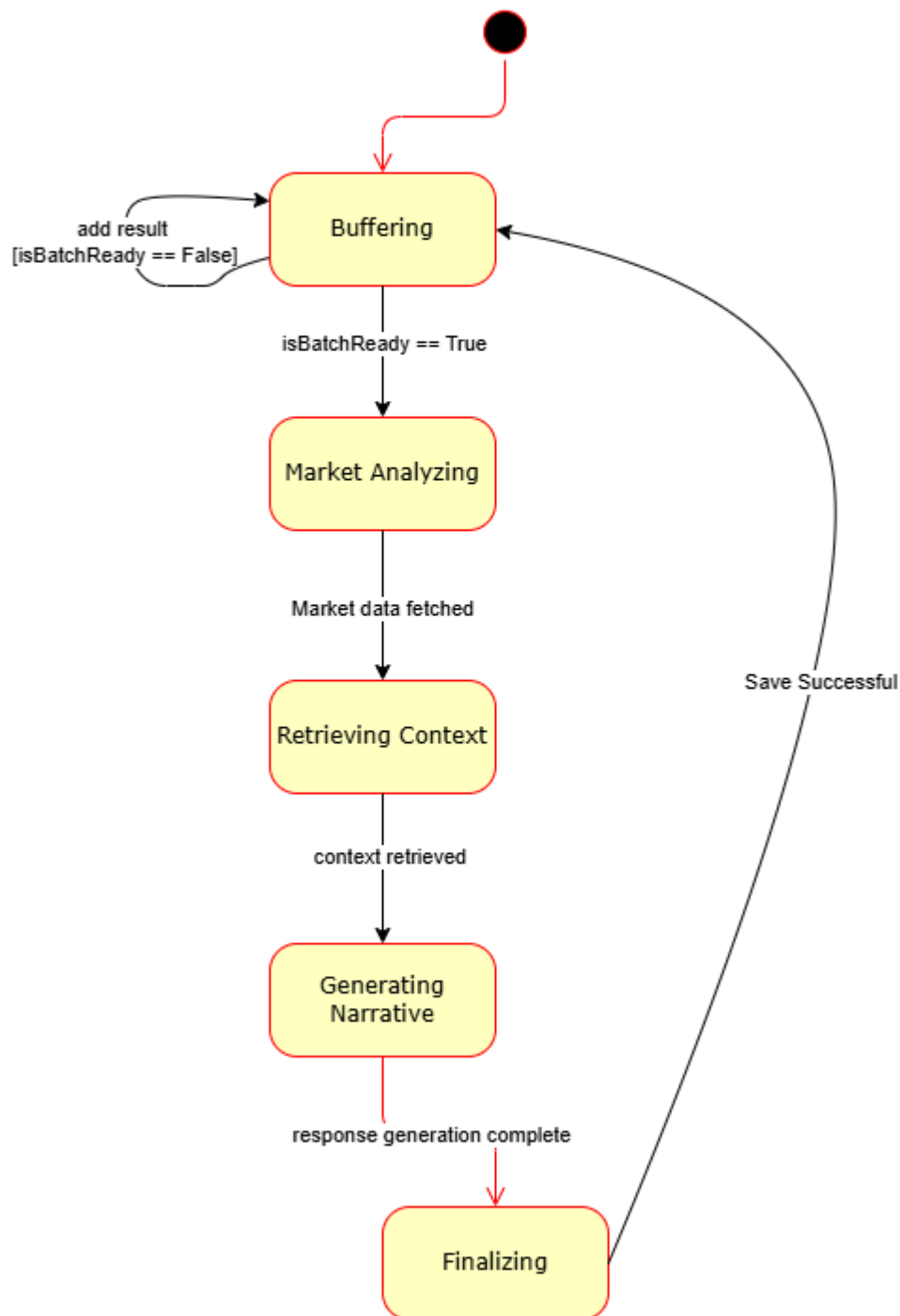


Figure 3.12: State Machine for Intelligence Orchestrator

3.5.4. Class Diagram

The Class Diagram below illustrates the object-oriented structure of ExportEdge AI across its four layers. The FrameExtractor in the Data Acquisition Layer interacts with the MotionDetector and ROIExtractor, passing frames to the

ImageProcessor. The ClassificationModel and SegmentationModel in the AI Analysis Layer execute inference and pass results to the ReportOrchestrator in the Intelligence Layer. The ReportOrchestrator coordinates the BatchManager, ExportDecisionEngine, and LLMService to generate and store export reports, which are then rendered by the Streamlit Frontend Layer.

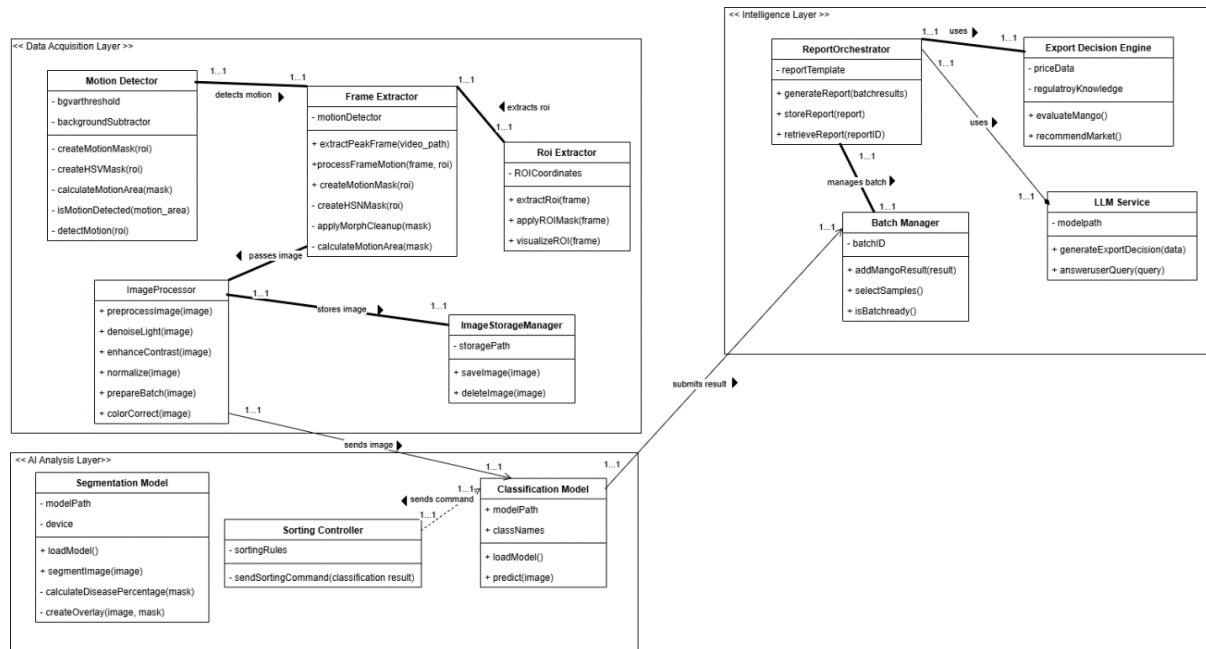


Figure 3.13: Class Diagram for ExportEdge

4. System Testing

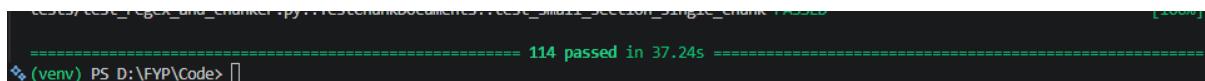
Extensive system testing has been conducted on ExportEdge AI to ensure that it performs optimally across all pipeline stages. We have divided the test cases into the following categories.

4.1. Unit Testing

We ensured that the individual components and functions of ExportEdge AI work correctly in unit testing.

- **Settings Defaults Validation:** All configuration attributes including CLASS_NAMES, INPUT_SHAPE, and HSV ranges are correctly defined with valid types and value ranges.
- **Global Settings Instance:** The module-level settings instance is correctly of type Settings and PROJECT_ROOT and CODE_DIR paths exist on the file system.
- **Image Preprocessing Shape:** All preprocessing functions correctly return output images with the same shape and uint8 dtype as the input.
- **Resize High Quality:** The function correctly resizes images to 150×150 pixels by default and preserves all three colour channels.
- **Image Postprocessing:** All postprocessing functions correctly preserve input shape, dtype, and pixel range including on small 32×32 images.
- **DefectRegion Construction:** The dataclass correctly stores all fields and mean_intensity defaults to 0.0 when not provided.
- **DefectAnalysis Construction:** The dataclass correctly stores all fields including defect count, colour uniformity score, and export grade impact.
- **MangoDefectDetector Scoring:** The detector correctly returns scores in range 0–100 and a valid export_grade_impact value of minimal, moderate, or significant.
- **Surface Quality Scoring:** The calculate_surface_quality_score function correctly returns 100.0 when no defect regions are present.
- **Export Grade Mapping:** The _map_to_grade function correctly assigns Grade A below 2% defects, Grade B between 2–5%, and Grade C above 5% or for significant impact.
- **LLM Output Parsing:** The parse_structured_recommendation function correctly returns four keys and correctly parses countries, actionable steps, and conditions from structured LLM output.
- **Export Advisor Metadata:** The build_metadata function correctly returns all eight expected keys and defaults export_grade to C for empty input.
- **Integrated Analyzer Grading:** The _generate_quality_assessment function correctly assigns Grade A, B, C, and Processing grades and returns overall_score in range 0–100.

- **Export Market Suitability:** The `_predict_export_suitability` function correctly includes USA for Grade A, Middle East for Grade B, and returns a non-empty `feature_description` string.
- **Report Generator Helpers:** The `_img_to_b64` function correctly returns a valid base64 PNG string and `_ns` correctly converts dictionaries to namespaces including nested structures.
- **Segmentation Overlay:** The `create_segmentation_overlay` and `get_disease_statistics` functions correctly return valid shapes and report zero regions for an empty mask.
- **Pipeline Factory:** The `PipelineFactory` correctly registers and creates pipelines with case-insensitive lookup and raises `ValueError` for unknown fruit types.
- **Abstract Pipeline Interface:** The `process_single_frame` correctly returns classification results and `process_video` correctly raises `RuntimeError` without setup.
- **SOP File Availability:** All SOP markdown files are confirmed present and non-empty in the `data_for_llm` directory.
- **Document Chunking:** The `chunk_documents` function correctly produces more than 20 non-empty chunks from seven SOP files with all source filenames represented.
- **Chunk Metadata:** Every chunk correctly carries all seven required metadata keys with valid types and `chunk_size` correctly matches `page_content` length.
- **Chunk ID Integrity:** Chunk IDs correctly start at zero, are globally sequential, unique, and total count equals maximum ID plus one.
- **Chunk Size Constraints:** No chunks exceed the defined size budget and no degenerate chunks under three characters exist.



```

===== 114 passed in 37.24s =====
(venv) PS D:\FYP\Code>

```

4.2. Integration Testing

The purpose of integration testing was to test the interactions between components and ensure data flows correctly across pipeline stages.

- **Defect to Export Grade Flow:** A synthetic mango image correctly passes through `MangoDefectDetector` and into `ExportAdvisor.build_metadata` producing a valid export grade.
- **Integrated Analyzer End-to-End:** The `analyze_mango_comprehensive` function correctly returns all five required keys and handles missing segmentation input with `ml_disease` as `None`.
- **JSON Report Schema:** A simulated inspection result correctly serializes to a valid JSON structure with all required fields per mango entry.
- **Multi-Document Chunking:** The `chunk_documents` function correctly produces chunks from multiple source documents with correct source metadata and non-empty content.

- **Preprocessing to Postprocessing Round-Trip:** A combined pipeline of preprocess_image, contrast, and smooth_image correctly preserves image shape, dtype, and pixel range.
- **Prompt Template Validation:** The prompt_templates.json correctly parses as valid JSON with all required keys including system_message, export_recommendation_prompt, and rag_config sub-keys.
- **Vision Pipeline Integration:** The process_single_frame correctly returns all required keys, class_name in the known class list, and disease_percentage matching the segmentation stub value of 5.0.
- **Segmentation Disabled:** With segmentation disabled, disease_percentage is correctly 0.0 and segmentation_mask is correctly None throughout the pipeline.
- **predict_top_k Integration:** The function correctly returns k results sorted in descending confidence order with all confidences in range 0–1.
- **Grade Boundary Testing:** The _map_to_grade function correctly assigns Grade A at exactly 2% defect boundary, Grade B above 2.1%, and Grade C above 5%.
- **Build Metadata Chain:** The build_metadata function correctly stores disease_percentage and derives grades consistent with _map_to_grade called with the same parameters.
- **FAISS Vector Store:** The FAISS index is correctly built from documents, retriever returns Document objects respecting the k limit, and the index correctly saves and reloads from disk.
- **LLM Context Formatting:** The format_context function correctly produces a non-empty string with source headers and page content, and returns empty string for empty input.
- **RAG Query Integration:** The rag_query function correctly returns success status, mocked LLM answer, sources as a list, and positive num_retrieved and context_length values.
- **Export RAG Query:** The export_rag_query function correctly returns success status, echoes inspection metadata, embeds export grade in the query, and handles retriever errors gracefully.
- **Full End-to-End Integration:** The complete chain from raw frame through defect detection, export advisory, and mocked RAG query correctly returns a valid export grade and non-empty answer.

```
===== 64 passed, 23 warnings in 20.34s =====
(venv) PS D:\FYP\Code> |
```

4.3. System Testing

We ensured that the entire system workflow from video upload to PDF report download works smoothly.

- **Healthy Mango Full Pipeline:** A healthy mango video correctly passes through the full pipeline producing a Healthy classification, Grade A, UN Comtrade price table, and a positive export recommendation.

- **Diseased Mango Full Pipeline:** A diseased mango video correctly produces a red segmentation overlay, computed disease percentage, and export advisory flagging the mango as ineligible.
- **Export Recommendation Generation:** The RAG pipeline correctly retrieves regulatory context and generates country-specific recommendations with actionable steps for the assigned grade.
- **PDF Report Download:** After inspection, the generated PDF correctly contains the timestamp, ROI image, segmentation overlay, quality metrics, price table, and LLM recommendation.
- **JSON Inspection Log Creation:** A structured JSON log is correctly saved to disk after each inspection containing all required fields.
- **RAG Chat with Export Questions:** After report embedding, the user correctly receives a relevant response with source citations when submitting a regulatory question via chat.
- **Quick Prompt Button Functionality:** Pre-defined prompt buttons on the chat page correctly populate the input and generate relevant LLM responses.
- **Multi-Mango Video Processing:** A video with multiple mangoes correctly produces individual result cards each with its own classification, segmentation overlay, and export grade.

4.4. Performance Testing

We ensured that the application runs efficiently on the Raspberry Pi 5 edge device under normal operating conditions.

- **Defect Detection Single Frame:** The detect_defects function completed within the 1500ms latency budget per frame on the edge device.
- **Defect Detection Average Latency:** The average latency over 10 consecutive calls remained within 1200ms with stable performance across runs.
- **Export Metadata Construction:** The build_metadata function completed within 50ms and one thousand _map_to_grade calls completed within 100ms total.
- **Full Defect to Metadata Chain:** The combined detect_defects and build_metadata pipeline completed within the combined budget of 550ms.
- **FAISS Vector Store Build:** Building a FAISS index from five documents completed within the 1000ms startup budget.
- **FAISS Single Query:** A single retriever query completed within 3000ms after warm-up, meeting the real-time RAG response requirement.
- **Context Formatting:** The format_context function completed within 100ms for a five-document context.
- **Memory Footprint:** Five MangoDefectDetector instances increased RSS by less than 100MB.

```
===== 13 passed, 3 warnings in 39.41s =====  
(venv) PS D:\FYP\Code> |
```

5. Deployment and System Integration

5.1. Deployment Details

ExportEdge AI was deployed on a Raspberry Pi 5 edge device using a headless setup with auto-start capability, making the system accessible from any browser on the local network without requiring manual intervention.

Component	Platform
Edge Hardware	Raspberry Pi5
Storage	Samsung Pro Endurance 64GB microSD
Frontend	Streamlit
Vector Database	Faiss DB (Embedded, local)
Database	SQLite
LLM	Groq – Llama (Open-AI compatible)
Embedding Model	all-MiniLM-L6-v2 (local)

Table 1: Deployment Details for ExportEdge

5.2. System Integration Details

Below is the integration flow of how all components interact within the ExportEdge AI platform:

- The user accesses the Streamlit web interface from any browser on the local network at http://<ip_address>:8501. The application auto-starts on Raspberry Pi boot via the exportededge.service systemd unit.
- The user uploads a video file or connects to the Dahua IP camera RTSP stream. The Data Acquisition Layer uses OpenCV to capture frames, apply HSV-based motion detection, and extract the peak-motion ROI frame containing the mango.
- The extracted ROI frame is preprocessed and passed to the TFLite MobileNetV2 classification model via tflite-runtime 2.14.0. The classification result including label and confidence score is returned and displayed in the Streamlit interface.
- The same ROI frame is passed to the TFLite DeepLabV3 segmentation model which generates a pixel-level disease mask. The mask is overlaid on the original image as a red highlight and the disease area percentage is computed and displayed.
- The OpenCV defect analysis module computes the surface quality score, colour uniformity score, and export grade from the classification and segmentation outputs.
- The Intelligence Layer fetches live mango export price data from the UN Comtrade API and displays a country-wise price table. The all-MiniLM-L6-v2 embedding model retrieves relevant regulatory context from the local FAISS vector store.
- The retrieved context along with inspection results is passed to the Groq API which returns a natural language export recommendation via the LLaMA LLM. The recommendation is displayed in the Streamlit interface.

- Upon PDF report generation, xhtml2pdf compiles all inspection results including the ROI image, segmentation overlay, quality metrics, price table, and LLM recommendation into a downloadable PDF file.
- The generated report content is automatically embedded into the FAISS vector store using the all-MiniLM-L6-v2 model, making it immediately available for follow-up queries via the RAG-based chat interface.
- User chat queries are forwarded to the RAG pipeline which retrieves relevant context from the FAISS vector store including the current session's embedded report, and the Groq LLM generates a contextual response displayed in the chat interface.

6. User Interfaces

6.1. Landing Page

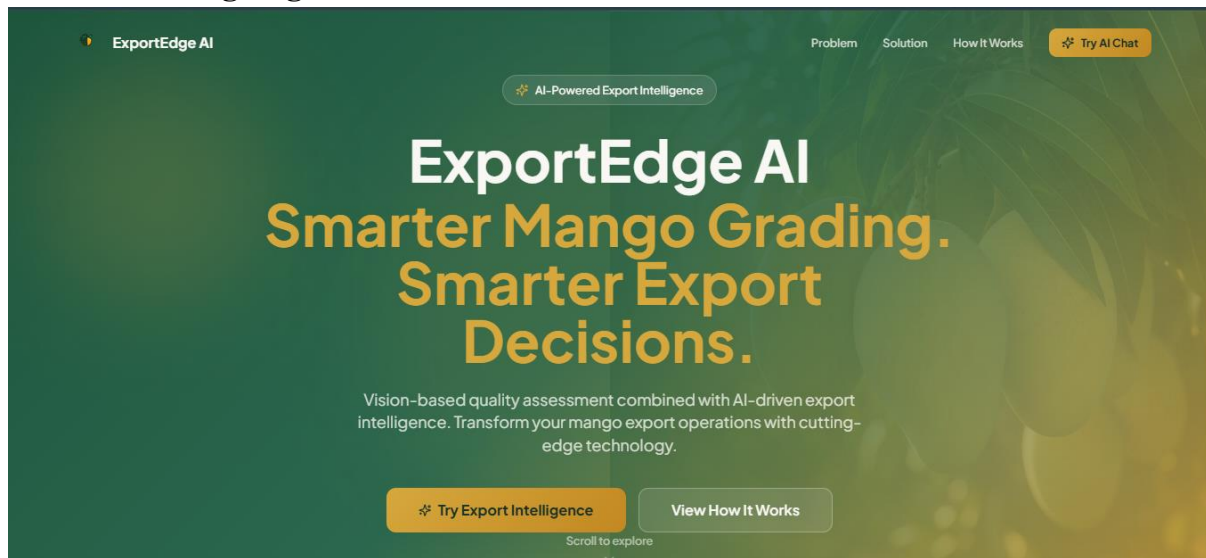


Figure 6.1: Landing page for ExportEdge

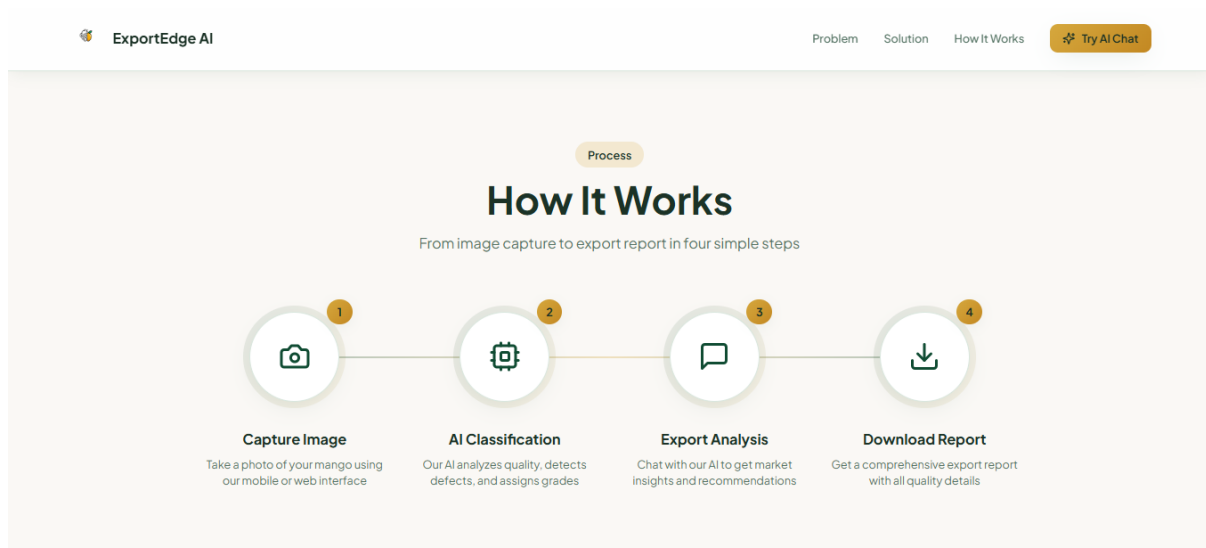


Figure 6.2: Landing Page (continued) for ExportEdge

6.2. Home Page

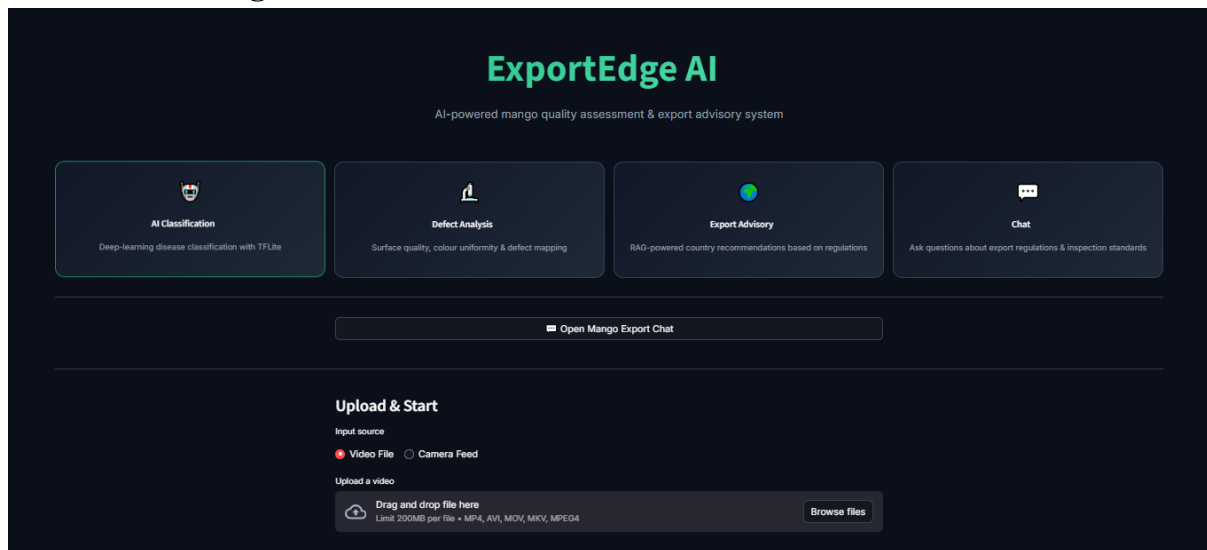


Figure 6.3: Home Page for ExportEdge

6.3. Processing Page

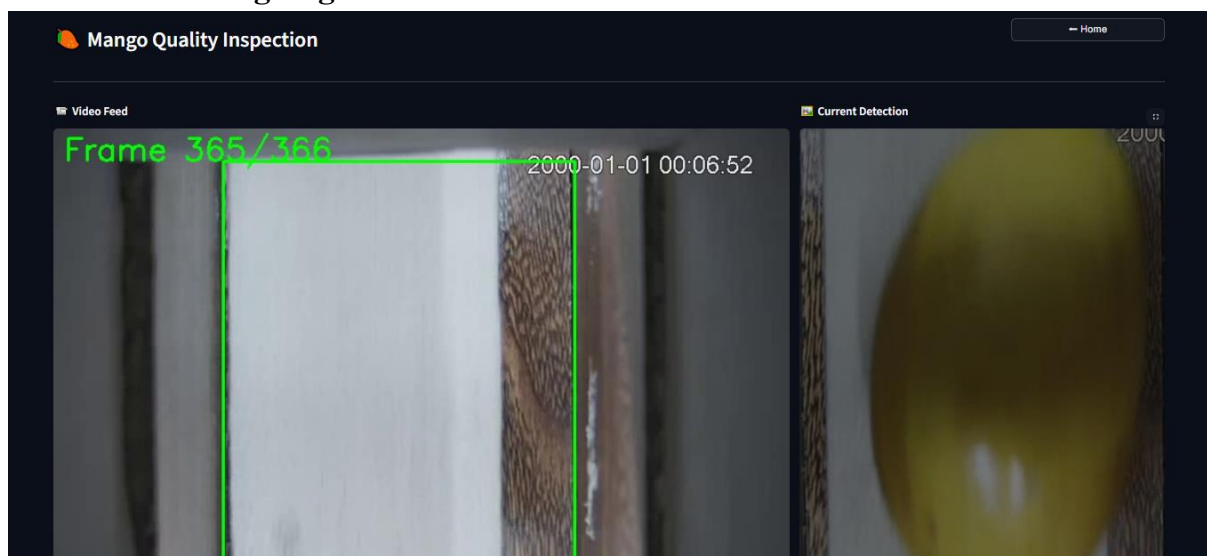


Figure 6.4: Processing Page for ExportEdge

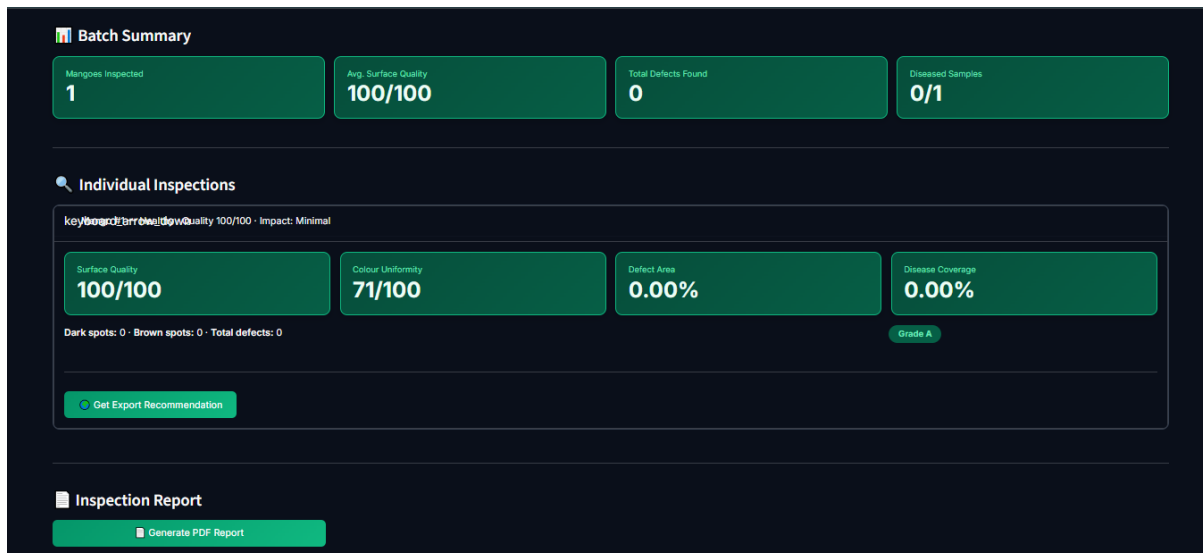


Figure 6.5: Processing Page (results) for ExportEdge

6.4. Chatbot Page

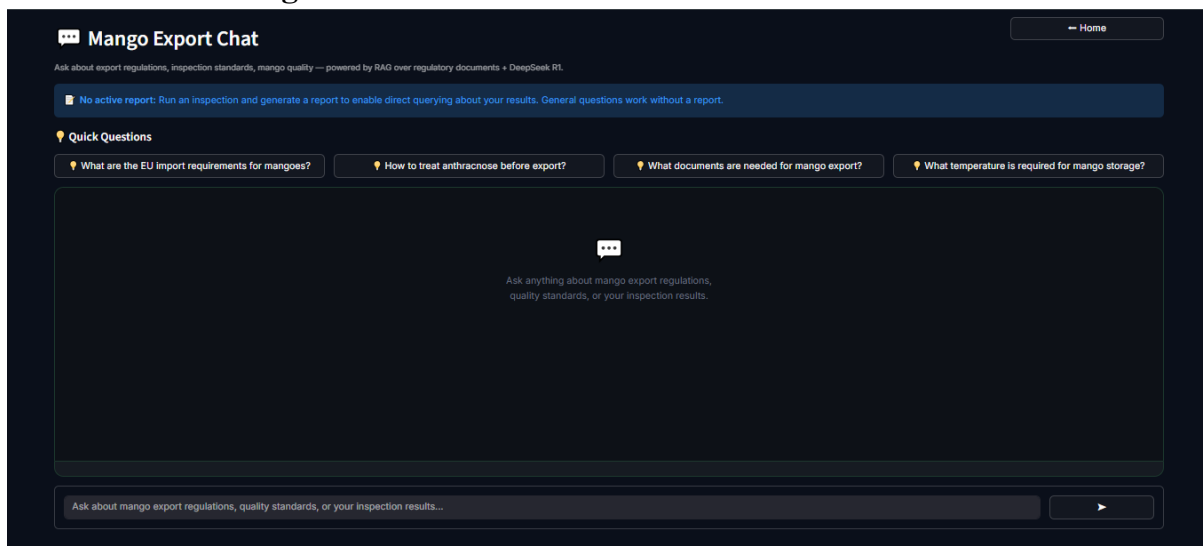


Figure 6.6: Chatbot Page for ExportEdge

7. Conclusion

ExportEdge AI is an edge-deployed agri-tech platform designed to transform the mango export industry in Pakistan by automating quality inspection and providing AI-driven export intelligence. Developed by undergraduate students at NUST-SEECs, the project addresses two critical challenges facing Pakistani mango exporters: inconsistent manual quality grading that leads to post-harvest losses of 25–40%, and the absence of real-time market intelligence to guide export decisions. By deploying a complete AI pipeline on a Raspberry Pi 5 edge device, ExportEdge AI brings automated inspection directly to the packhouse where quality decisions are made.

The system integrates a TFLite MobileNetV2 classification model for disease detection, a TFLite DeepLabV3 segmentation model for disease region mapping, and an OpenCV-based defect analysis module for surface quality grading, all running locally on the edge device without cloud dependency. Classification and segmentation inference complete within 2 seconds approximately on the Raspberry Pi 5, meeting the real-time requirements of a packhouse environment. The AI Analysis Layer is complemented by an Intelligence Layer that fetches live mango export price data from the UN Comtrade API, retrieves relevant regulatory context from a local FAISS vector store using the all-MiniLM-L6-v2 sentence embedding model, and generates natural language export recommendations via the Groq-hosted LLaMA large language model through a Retrieval-Augmented Generation pipeline.

The system features a Streamlit web interface accessible from any browser on the local network, providing real-time inspection feedback, export advisory dashboards, an interactive RAG-based chat interface, and automated PDF report generation. The application auto-starts on Raspberry Pi boot via a systemd service, requiring no technical intervention from packhouse operators. A Dahua IP camera connected via ethernet provides the live video feed, with OpenCV handling motion detection and peak-frame extraction from the conveyor belt stream. Generated inspection reports are automatically embedded into the FAISS vector store, enabling users to ask follow-up questions about specific inspection results through the chat interface.

Comprehensive testing was conducted across unit, integration, system, performance, and security categories using pytest 9.0.2 with synthetic data and mocked external dependencies. The system demonstrated stable memory usage over extended operation, correct sequential data flow across all pipeline layers, and secure handling of API keys and model files. The modular object-oriented architecture ensures that individual components, classification, segmentation, defect analysis, and export advisory, can be independently maintained, retrained, or extended to support additional fruit types without restructuring the core system.

ExportEdge AI directly contributes to United Nations Sustainable Development Goal 8 (Decent Work and Economic Growth) by supporting Pakistan's agricultural export economy through automated quality inspection and data-driven market intelligence, enabling exporters to maximize revenue and reduce cargo rejections at destination ports. The system also contributes to SDG 12 (Responsible Consumption and Production) by promoting more efficient and responsible use of agricultural produce through accurate AI-based grading, reducing post-harvest waste and ensuring that only export-quality mangoes enter the supply chain.

Future work includes adding more cameras and scanning mangoes from all sides, retraining the models on a larger and more diverse mango disease dataset to improve accuracy, extending the system to support additional fruit types through the modular pipeline factory architecture, and exploring quantization-aware training to further reduce TFLite model inference latency on the edge device and integrating more SOP related documents in the RAG system for better retrieval precision and LLM responses.

8. References

- [1] United Nations, *Sustainable Development Goals (SDG 8, SDG 9, SDG 12)*. United Nations, 2023. [Online]. Available: <https://sdgs.un.org/goals>
- [2] M. Saleem and N. Ahmad, "Quality Grading and Ripeness Detection of Mangoes Using Computer Vision and Deep Learning," *Computers and Electronics in Agriculture*, 2022.
- [3] D. Zhang, "Fruit 360 Classification Based on the Convolutional Neural Network," *Applied and Computational Engineering*, vol. 15, pp. 219–222, 2023. [Online]. Available: <https://www.ewadirect.com/proceedings/ace/article/view/4578>
- [4] K. Munir, U. Iqbal, and W. Yousaf, "Automatic Fruits Classification System Based on Deep Neural Network," *NUST Journal of Engineering Sciences*, vol. 13, no. 1, 2023. [Online]. Available: <https://journals.nust.edu.pk/index.php/njes/article/view/501>
- [5] Raspberry Pi Foundation, *Raspberry Pi Documentation: Machine Learning on Edge Devices*. 2024. [Online]. Available: <https://www.raspberrypi.com/documentation/>
- [6] Hugging Face, *Sentence Transformers: all-MiniLM-L6-v2*. 2024. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [7] Google, *TensorFlow Lite , Machine Learning for Mobile and Edge Devices*. 2024. [Online]. Available: <https://www.tensorflow.org/lite>
- [8] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking Atrous Convolution for Semantic Image Segmentation," *arXiv:1706.05587*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.05587>
- [9] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [10] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [11] United Nations Statistics Division, *UN Comtrade Database , International Trade Statistics*. 2024. [Online]. Available: <https://comtradeplus.un.org/>
- [12] Meta AI, "The Llama 3 Herd of Models," *arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>